MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Vasilean Vitalie
Group FAF-241

Report

Laboratory Work No. 1
Operating Systems
Topic: The OS as a Resource Manager

Checked by:
univ. assist. / lect. Patricia Reițman
Department of Software Engineering and Automation
FCIM, UTM

Chisinau – 2026

Contents

1 INTRODUCTION AND GOAL **3**

1.1 Introduction 3

1.2 Lab Objective 3

2 PART 1: FILES AND DIRECTORIES **4**

2.1 Command Execution and Raw Output 4

2.2 Analysis and Observations 4

3 PART 2: PROCESSES **6**

3.1 Command Execution and Raw Output 6

3.2 Analysis and Observations 6

4 PART 3: MEMORY **7**

4.1 Command Execution and Raw Output 7

4.2 Analysis and Observations 7

5 PART 4: DEVICES AND STORAGE **8**

5.1 Command Execution and Raw Output 8

5.2 Analysis and Observations 8

6 CONCLUSIONS **9**

1 INTRODUCTION AND GOAL

1.1 Introduction

An operating system fundamentally functions as a resource manager and extended machine abstraction. It arbitrates and virtualizes access to underlying hardware components: processor time, physical and virtual memory, persistent storage systems, and peripheral devices. Through standard POSIX system interfaces, user processes interact with hardware indirectly, allowing the operating system to enforce process isolation, access control, memory safety, and equitable scheduling.

1.2 Lab Objective

The primary objective of this laboratory work is to examine how modern Linux manages core hardware and software resources through direct command-line utilities and kernel interfaces. The specific scope comprises:

- inspecting filesystem organization, file permissions, ownership metadata, and root directory hierarchies;
- examining process states, process identification (PID), lifecycle control via signals, and process metadata via the `/proc` pseudo-filesystem;
- evaluating virtual memory allocations, resident set size (RSS), system memory limits, and swap utilization;
- inspecting storage device blocks, mount points, filesystem capacities, and hardware device abstractions exposed under `/dev`.

2 PART 1: FILES AND DIRECTORIES

2.1 Command Execution and Raw Output

The environment was set up by inspecting system identity and manipulating files within a dedicated workspace:

```
vitalie@Lenovo:~$ whoami ; uname -a ; uptime
vitalie
Linux Lenovo 6.6.87.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18:31:42 UTC 2025
     x86_64 x86_64 x86_64 GNU/Linux
 17:42:20 up 5 min,  1 user,  load average: 0.62, 0.26, 0.09

vitalie@Lenovo:~$ mkdir -p ~/os-lab1/demo && cd ~/os-lab1/demo
vitalie@Lenovo:~/os-lab1/demo$ echo "hello operating systems" > note.txt
vitalie@Lenovo:~/os-lab1/demo$ cp note.txt copy.txt
vitalie@Lenovo:~/os-lab1/demo$ mv copy.txt renamed.txt
vitalie@Lenovo:~/os-lab1/demo$ rm renamed.txt
vitalie@Lenovo:~/os-lab1/demo$ ls -l note.txt
-rw-r--r-- 1 vitalie vitalie 24 Sep 28 17:43 note.txt

vitalie@Lenovo:~/os-lab1/demo$ chmod 600 note.txt
vitalie@Lenovo:~/os-lab1/demo$ ls -l note.txt
-rw------- 1 vitalie vitalie 24 Sep 28 17:43 note.txt

vitalie@Lenovo:~/os-lab1/demo$ chmod 644 note.txt
vitalie@Lenovo:~/os-lab1/demo$ ls -l note.txt
-rw-r--r-- 1 vitalie vitalie 24 Sep 28 17:43 note.txt
```

Figure 2.1: Figure 1.1 – Execution of system identification and file permission manipulation

The root filesystem was then inspected to examine standardized top-level directories:

```
vitalie@Lenovo:~$ ls -la /
total 2748
drwxr-xr-x  23 root root    4096 Sep 28 17:37 .
drwxr-xr-x  23 root root    4096 Sep 28 17:37 ..
lrwxrwxrwx   1 root root       7 Apr 22  2024 bin -> usr/bin
drwxr-xr-x  16 root root    4080 Sep 28 17:37 dev
drwxr-xr-x  93 root root    4096 Sep 28 17:41 etc
drwxr-xr-x   3 root root    4096 Sep 19  2025 home
dr-xr-xr-x 253 root root       0 Sep 28 17:37 proc
drwxr-xr-x  14 root root    4096 Apr 22  2024 usr
drwxr-xr-x  13 root root    4096 Apr 22  2024 var
```

Figure 2.2: Figure 1.2 – Standard root directory layout in Linux

2.2 Analysis and Observations

a) **Ownership and groups:** The file `note.txt` is owned by the user `vitalie` and associated with group `vitalie`.

b) **Permission notation:** The string `-rw-------` indicates a regular file (`-`) where the owner possesses read and write permissions (`rw-`), while the group (`---`) and others (`---`) have zero access permissions.

c) **Role of core root directories:**

- `/etc`: Houses system-wide static configuration files and service configurations.
- `/home`: Contains personal directories and userland files for standard accounts.
- `/dev`: Contains special device node files representing hardware and virtual peripherals.

3 PART 2: PROCESSES

3.1 Command Execution and Raw Output

Running processes, tree relationships, and asynchronous job controls were examined:

```
vitalie@Lenovo:~$ ps aux | head -n 5
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.0   2680  1664 ?        Ss   17:37   0:00 /init
root           6  0.0  0.0   2684   220 ?        S    17:37   0:00 /init
vitalie        7  0.0  0.0  10668  5248 pts/0    Ss   17:37   0:00 -bash
vitalie      120  0.0  0.0  11824  3456 pts/0    R+   17:44   0:00 ps aux

vitalie@Lenovo:~$ ps aux | wc -l
28

vitalie@Lenovo:~$ sleep 300 &
[1] 3929
vitalie@Lenovo:~$ jobs
[1]+  Running                 sleep 300 &
vitalie@Lenovo:~$ ps -ef | grep sleep
vitalie     3929    7  0 17:45 pts/0    00:00:00 sleep 300
vitalie     3931    7  0 17:45 pts/0    00:00:00 grep --color=auto sleep

vitalie@Lenovo:~$ cat /proc/3929/status | head -n 10
Name:   sleep
Umask:  0022
State:  S (sleeping)
Tgid:   3929
Ngid:   0
Pid:    3929
PPid:   7
TracerPid:      0
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000

vitalie@Lenovo:~$ kill 3929
[1]+  Terminated              sleep 300
```

Figure 3.1: Figure 1.3 – Background process creation, state inspection via /proc, and termination

3.2 Analysis and Observations

a) **Process count and top resource consumer:** On the minimal WSL2 guest, 28 processes were active. The primary CPU/memory consumer during command execution is the shell and `top`/`ps` itself.

b) **Process state of sleep:** The `State:` field of `/proc/3929/status` indicates `S (sleeping)`, representing an interruptible sleep state where the process yields execution while awaiting a timer interrupt.

c) **Job control vs PID targeting:** The shell job specifier `%1` refers to the job index managed in the shell's internal table, whereas `3929` is the global Process Identifier (PID) recognized system-wide by the kernel.

4 PART 3: MEMORY

4.1 Command Execution and Raw Output

Physical RAM, kernel memory counters, and individual process memory footprints were evaluated:

```
vitalie@Lenovo:~$ free -h
               total        used        free      shared  buff/cache   available
Mem:           7.5Gi       356Mi       7.1Gi       2.0Mi        98Mi       7.0Gi
Swap:          2.0Gi          0B       2.0Gi

vitalie@Lenovo:~$ cat /proc/meminfo | head -n 6
MemTotal:        7889508 kB
MemFree:         7431200 kB
MemAvailable:    7341856 kB
Buffers:           14200 kB
Cached:            86112 kB
SwapCached:            0 kB

vitalie@Lenovo:~$ sleep 300 & PID=$!
[1] 4015
vitalie@Lenovo:~$ grep VmRSS /proc/$PID/status
VmRSS:      1536 kB
vitalie@Lenovo:~$ kill $PID
[1]+  Terminated              sleep 300
```

Figure 4.1: Figure 1.4 – Memory allocation inspection using free, /proc/meminfo, and VmRSS

4.2 Analysis and Observations

a) **Total and available physical memory:** The virtual environment has 7.5 GiB of total RAM assigned, with approximately 7.1 GiB unallocated (free).

b) **Swap partition:** The system has 2.0 GiB of swap space allocated, configured to page out anonymous inactive pages if physical memory experiences severe pressure.

c) **Resident memory footprint of `sleep`:** The resident memory size (`VmRSS`) of a clean `sleep` process is 1536 kB. This represents the actual physical memory pages mapped for code segments, dynamically linked C standard libraries (`libc.so`), and process runtime stack.

5 PART 4: DEVICES AND STORAGE

5.1 Command Execution and Raw Output

Disk partitions, block devices, storage utilization, and pseudo-device entries were inspected:

```
vitalie@Lenovo:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdc       1007G  3.1G  953G   1% /
none            3.8G     0  3.8G   0% /usr/lib/wsl/drivers
none            3.8G     0  3.8G   0% /dev/shm
none            3.8G  4.0K  3.8G   1% /run
C:\             476G  312G  165G  66% /mnt/c

vitalie@Lenovo:~$ lsblk
NAME   MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda      8:0    0 388.3M  1 disk
sdb      8:16   0     2G  0 disk [SWAP]
sdc      8:32   0     1T  0 disk /

vitalie@Lenovo:~$ du -sh ~/os-lab1
36K     /home/vitalie/os-lab1

vitalie@Lenovo:~$ ls -l /dev | head -n 8
total 0
crw-rw-rw- 1 root root   1,   3 Sep 28 17:37 null
crw-rw-rw- 1 root root   1,   5 Sep 28 17:37 zero
crw-rw-rw- 1 root root   1,   7 Sep 28 17:37 full
crw-rw-rw- 1 root root   1,   8 Sep 28 17:37 random
crw-rw-rw- 1 root root   1,   9 Sep 28 17:37 urandom
crw-rw-rw- 1 root tty    5,   0 Sep 28 17:37 tty
```

Figure 5.1: Figure 1.5 – Disk partitioning, storage consumption, and device nodes under /dev

5.2 Analysis and Observations

a) **Root device partition:** The root directory (`/`) is mounted from the block device `/dev/sdc`, having a virtual capacity of 1007 GB with only 1% (3.1 GB) utilized.

b) **Device node classification in `/dev`:**

- `/dev/tty`: Character device (`c`) representing the active controlling terminal interface.
- `/dev/sdc`: Block device (`b`) representing secondary persistent storage, operating with block-level cache buffering.

c) **Everything is a file philosophy:** UNIX and Linux abstract varied hardware interfaces (hard disks, pseudo-random generators, consoles) through uniform VFS file nodes under `/dev`, allowing userland programs to utilize standard POSIX system calls (`open`, `read`, `write`, `close`) without custom device drivers.

6 CONCLUSIONS

In this laboratory work, the foundational principles of the Linux operating system as a comprehensive hardware and software resource manager were evaluated.

By experimenting with file manipulation, ownership, and permission masks, the mechanics of Discretionary Access Control (DAC) were verified. Investigating process trees, background execution, and the `/proc` filesystem demonstrated how the Linux kernel exposes live runtime statistics and thread states directly to userland utilities. Furthermore, memory metrics obtained from `free` and `/proc/meminfo` highlighted the relationship between virtual address space and physical Resident Set Size (RSS). Finally, inspecting block storage, mount topologies, and character devices in `/dev` confirmed the practical realization of the fundamental UNIX paradigm that unifies heterogeneous system resources under a common hierarchical filesystem interface.